Cyberscope

A TAC Security Company

Audit Report

EZManager

February 2026

Repository https://github.com/EZManagerCL/EZManagerContracts

Commit deca794869fc7f5d2ce4bda3ea2b00a53782f975

Audited by © cyberscope

Table of Contents

Table of Contents **1**
Risk Classification **3**
Review **4**
Audit Updates 4
Source Files 5
Overview **6**
CLCore 6
Valuation 6
CLManager 7
UniswapAdapter 8
AerodromeAdapter 8
RebalancePlanner 9
Findings Breakdown **10**
Diagnostics **11**
ARS - Agnostic Route Size 12
Description 12
Recommendation 13
Team Update 13
IPV - Inconsistent Position Valuation 14
Description 14
Recommendation 15
Team Update 15
CCR - Contract Centralization Risk 16
Description 16
Recommendation 18
Team Update 18
CPI - Cross-Tick Price Impact 19
Description 19
Recommendation 20
Team Update 20
IDLS - Ineffective Decrease Liquidity Slippage 21
Description 21
Recommendation 22
Team Update 22
PFO - Pool Fee Omission 23
Description 23
Recommendation 24
Team Update 24
PTWD - Possible TWAP Window Desync 25

Description 25
Recommendation 26
Team Update 26
PSM - Price Source Mismatch 27
Description 27
Recommendation 28
Team Update 29
STW - Short TWAP Window 30
Description 30
Recommendation 31
Team Update 31
SCRS - Stale Cached Route Scores 32
Description 32
Recommendation 33
Team Update 33
UQA - Unconstrained Quoted Amount 34
Description 34
Recommendation 35
Team Update 35
ZVE - Zero Value Estimation 36
Description 36
Recommendation 37
Team Update 37
L04 - Conformance to Solidity Naming Conventions 38
Description 38
Recommendation 39
Functions Analysis 40
Inheritance Graph 58
Flow Graph 59
Summary 60
Disclaimer 61
About Cyberscope 62

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Repository	https://github.com/EZManagerCL/EZManagerContracts
Commit	deca794869fc7f5d2ce4bda3ea2b00a53782f975
Explorer	https://basescan.org/address/0x9EfDe1640231A5306304317649f548bE73A5FA28 https://basescan.org/address/0x61c36AFF32Be348a3D1FE1E2B4745048f652770F https://basescan.org/address/0xB822F2a166960b85CF5AF56ECA6cB412e78fbd14 https://basescan.org/address/0x6e08CAb6ebB204aC7473aFdb09C6e914f4B4f749 https://basescan.org/address/0x06f655A7642Bb53cAa8723B2bF2410915B9799Be https://basescan.org/address/0x0e5d5ADE8E9f5ddc2df1CAeBc63EA37c1fC5be3B https://basescan.org/address/0xAe79B33073B8eE252de6B13a6dfF63ed3871a548

Audit Updates

Initial Audit	23 Dec 2025
Corrected Phase 2	4 Feb 2026
Corrected Phase 3	13 Feb 2026

Source Files

Filename	SHA256
Valuation.sol	d19a5a3ade2bc253c36d72ac158fbd4f35579107eba815eb879c4b4f9018cfe8
UniswapAdapter.sol	cac4cdea67751b55090e3d1594f6055a7b0eda4658db0e0a818237661ae29253
RebalancePlanner.sol	314a144c14d7df2418bbcb9a8cc51de424c885f628bf6e0829a5c16e6ce06869
CLManager.sol	48503fd9b6e401dd98f042a094ae9f40617121c882a2a2bfc4d979d5a8bf0167
CLCore.sol	54fff175273045a7940f8340f7b1878372ac1afcf1913aaf6c202762fc68a4b5
AerodromeAdapter.sol	9bea519e7c42117fdc152b7272453b070480261823994c02b876461f9620b647

Overview

The protocol implements a Concentrated Liquidity Management System for managing, accounting, and valuing liquidity positions across Uniswap V3 and Aerodrome Slipstream. The architecture separates state/accounting (CLCore) from pricing logic (Valuation), providing modular tracking of Liquidity Provider (LP) positions and their USDC-equivalent values.

CLCore

CLCore serves as the central ledger of the system. It is a contract responsible for tracking positions, managing balances, and calculating accrued fees.

Key Responsibilities:

- **Position Registry:** Maintains a mapping of Position structs that link a unique key to: LP NFT (tokenId), Owner address, Tick range (tickLower, tickUpper), Deposited principal (totalDepositedUSDC), Pending fees and dust balances.
- **Fee Accounting:** Calculates accrued fees for positions by comparing the pool's pending fees against the position's snapshots. This provides consistent fee tracking across Uniswap V3 and Aerodrome pools.
- **Dust Management:** Tracks residual USDC that is not currently deployed in a liquidity range, ensuring precise accounting of user balances.
- **Position Lifecycle Management:** Supports registration, deregistration, and updating of positions, including adjustments to total deposited USDC.

Valuation

Valuation is a view-only oracle contract that computes USDC-equivalent values for tokens held in the system.

Key Responsibilities:

- **USDC Valuation:** Converts token amounts into USDC-equivalent values.
- **Routing Logic:** Implements routing to identify the best pool based on liquidity depth scores. Supports direct USDC routing, and two-hop connector paths (via WETH or other bridge tokens).

- **Multi-DEX Support:** Handles both standard Uniswap V3 and Aerodrome pools, including different tick spacing and fee structures.
- **Liquidity Thresholds:** Ignores pools below a configurable minimum liquidity threshold.
- **TWAP Calculation:** Computes a short (default 60-second) Time-Weighted Average Price for price stability.
- **Route Caching:** Maintains cached edge scores for efficient route lookups, refreshed via `refreshAll()`.

CLManager

CLManager is the primary user-facing contract that orchestrates position operations. It coordinates between CLCore, Valuation, adapters, and the RebalancePlanner.

Key Responsibilities:

- **Position Opening:** Accepts USDC deposits, deducts protocol fees, seeds token pairs via adapters, plans rebalances, mints NFT positions, and registers them in CLCore.
- **Position Exiting:** Unwinds positions, converts tokens back to USDC, collects fees, and returns funds to users.
- **Fee Compounding:** Collects accrued trading fees and reinvests them into existing positions.
- **Collateral Management:** Supports adding and removing collateral from positions.
- **Range Changes:** Allows repositioning of liquidity to different tick ranges.
- **Bot Operations:** Supports automated operations via allowlisted bots with configurable bot fees.

Technical Highlights:

- **Second Pass Logic:** Attempts to deploy leftover USDC after initial minting if above configurable thresholds.
- **Slippage Budget Management:** Tracks remaining loss budget across multi-step operations.
- **Batch Operations:** Supports batch compounding and fee collection with configurable limits.

UniswapAdapter

UniswapAdapter is the DEX adapter implementation for Uniswap V3 and PancakeSwap V3 pools. It implements the `ICLDexAdapter` interface.

Key Responsibilities:

- **Position Minting:** Creates new concentrated liquidity positions via the NonfungiblePositionManager.
- **Liquidity Management:** Adds and removes liquidity from existing positions.
- **Token Swaps:** Executes swaps via the SwapRouter with TWAP-based slippage protection.
- **Fee Collection:** Collects accrued trading fees from positions.
- **Seed Pair Conversion:** Converts USDC into position tokens, supporting direct routes and bridge token swaps.

Technical Highlights:

- **TWAP-Based Slippage:** Uses pool TWAP for calculating expected outputs and minimum amounts.
- **Remaining Loss Budget Pattern:** Tracks cumulative slippage across multi-step operations.
- **PancakeSwap Support:** Includes `isPancakeSwap` flag to handle router interface differences.
- **Route Discovery:** Delegates to Valuation for best route selection based on cached depth scores.

AerodromeAdapter

AerodromeAdapter is the DEX adapter implementation for Aerodrome Slipstream pools on Base. It implements the `ICLDexAdapter` interface with Aerodrome-specific handling.

Key Responsibilities:

- **Position Minting:** Creates new concentrated liquidity positions via the Slipstream NonfungiblePositionManager.
- **Liquidity Management:** Adds and removes liquidity from existing positions.

- **Token Swaps:** Executes swaps via the Slipstream SwapRouter with TWAP-based slippage protection.
- **Fee Collection:** Collects accrued trading fees from positions.
- **Seed Pair Conversion:** Converts USDC into position tokens for Aerodrome pools.

Technical Highlights:

- **Tick Spacing vs Fee:** Aerodrome uses tick spacing instead of fee tiers for pool identification.
- **Slipstream Interfaces:** Uses Aerodrome-specific interfaces (ISlipstreamSwapRouter, ISlipstreamNonfungiblePositionManager).
- **TWAP Integration:** Computes TWAP via pool observations for price stability.

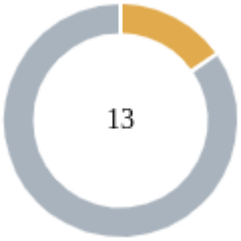
RebalancePlanner

RebalancePlanner computes optimal swap amounts to achieve the desired token ratio for a given tick range using iterative numerical methods.

Key Responsibilities:

- **Swap Amount Calculation:** Determines optimal swap direction and amount to balance token holdings for a target liquidity range.
- **Iterative Solver:** Uses a damped Newton's method (Iterative Value Decomposition) to converge on optimal amounts.
- **Price Impact Modeling:** Accounts for price movement during swaps when computing target ratios.

Findings Breakdown



Critical	0
Medium	2
Minor / Informative	11

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	2	0	0
Minor / Informative	0	11	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Medium	ARS	Agnostic Route Size	Acknowledged
Medium	IPV	Inconsistent Position Valuation	Acknowledged
Minor	CCR	Contract Centralization Risk	Acknowledged
Minor	CPI	Cross-Tick Price Impact	Acknowledged
Minor	IDLS	Ineffective Decrease Liquidity Slippage	Acknowledged
Minor	PFO	Pool Fee Omission	Acknowledged
Minor	PTWD	Possible TWAP Window Desync	Acknowledged
Minor	PSM	Price Source Mismatch	Acknowledged
Minor	STW	Short TWAP Window	Acknowledged
Minor	SCRS	Stale Cached Route Scores	Acknowledged
Minor	UQA	Unconstrained Quoted Amount	Acknowledged
Minor	ZVE	Zero Value Estimation	Acknowledged
Minor	L04	Conformance to Solidity Naming Conventions	Acknowledged

ARS - Agnostic Route Size

Criticality	Medium
Location	Valuation.sol#L219
Status	Acknowledged

Description

The `_bestRouteCached` function selects the route with the highest depth score (`scoreUSDC`), which reflects a pool's ability to absorb sell pressure without significant price impact. However, the actual token amount being evaluated is never compared against this depth capacity.

As a result, when a large token amount is routed through a pool with insufficient depth, the system still prices the entire amount using a flat time-weighted average price (TWAP). This approach ignores the potential for substantial slippage, leading to inaccurate valuations. Although depth data is available, it is used solely for ranking routes and not for assessing whether a route can realistically handle the trade size.

```shell
function usdcValue(address dex, address token, uint256
amount) external view returns (uint256) {
    if (amount == 0) return 0;
    if (token == USDC) return amount;
    if (CORE == address(0)) revert CoreNotSet();
    if (!ICLCore(CORE).allowedDexes(dex)) revert
DexNotAllowed();
    DexContext memory ctx = _ctx(dex);
    if (ctx.factory == address(0)) revert
DexFactoryNotFound();

    (address poolA, address poolB, ) =
_bestRouteCached(dex, token, USDC);
    if (poolA == address(0)) revert RouteNotFound();
```

```
    uint256 outScaled = _quoteRoute(poolA, poolB, token,
USDC, amountScaled);

}
```

Recommendation

The team could consider validating that the amount being evaluated does not significantly exceed the route's depth score, or applying a discount factor to amounts that surpass the pool's capacity. This approach can help ensure more accurate pricing and prevent unrealistic valuations in scenarios where liquidity is insufficient.

Team Update

The team has acknowledged that this is not a security issue and states:

Valuation returns a mid-price estimate and intentionally avoids on-chain quoter or swap simulation to remain cheap and deterministic, so it does not model swap fees, price impact, or tick-crossing. It is used for informational and accounting purposes and for coarse sizing or gating decisions (for example, computing a fee basis). Valuation is not treated as an executable quote.

IPV - Inconsistent Position Valuation

Criticality	Medium
Location	CLCore.sol#L839,1051
Status	Acknowledged

Description

The `_positionValueUSDCSingle` function calculates the value of a position using two different pricing mechanisms, which may lead to inconsistencies during periods of high market volatility.

```shell
function _positionValueUSDCSingle(Position memory p)
internal view returns (uint256) {
    if (p.owner == address(0) || p.tokenId == 0) revert
PositionNotFound();

    (uint256 amt0, uint256 amt1, , ) =
_spotAmountsForPosition(p);

    uint256 v0 = _valueTokenToUSDC(p.dex, p.token0, amt0);
    uint256 v1 = _valueTokenToUSDC(p.dex, p.token1, amt1);
    return v0 + v1 + p.dustUSDC;

}
```

Within `_spotAmountsForPosition`, the function derives token amounts using the spot price obtained from `_readSlot0`:

```shell
(uint160 sqrtPriceX96,) = _readSlot0(pool);

(amt0, amt1) =
LiquidityAmounts.getAmountsForLiquidity(
    sqrtPriceX96,
    sqrtRatioAX96,
    sqrtRatioBX96,
    liquidity

);
```

However, when converting these token amounts to USDC, the `_valueTokenToUSDC` function relies on the `VALUATION.usdcValue` method, which uses a time-weighted average price (TWAP). This mismatch can result in inconsistent or misleading position values.

Recommendation

The team is advised to use a consistent price source throughout the entire position valuation calculation. This ensures uniformity in pricing data, reduces the risk of discrepancies, and improves the accuracy and reliability of the valuation process.

Team Update

The team has acknowledged that this is not a security issue and states:

The amounts for positions are intended to reflect the real on-chain state, so _spotAmountsForPosition uses slot0. These amounts are then valued using TWAP for manipulation resistance. This is an intentional choice. Token amounts in the position should reflect true on-chain amounts while the valuation of those amounts is based on TWAP.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	CLCore.sol#L161,170,176,183,192,217,254,260,267,303,328,334,341,348,352 CLManager.sol#L213,219,225,231,237,243,249,253 UniswapAdapter.sol#L133,137,141,147,153,161,167 AerodromeAdapter.sol#L124,128,132,138,144,152,158 Valuation.sol#L73,81,87,97 RebalancePlanner.sol#L113,120,125,131
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```shell
function setValuation(address valuation) external onlyOwner {...}
function setProtocolReserve(address reserve) external onlyOwner {...}
function setManager(address mgr) external onlyOwner {...}
function setGuardian(address g) external onlyOwner {...}
function addAllowedDex(address dex) external onlyOwner {...}
function removeAllowedDex(address dex) external onlyOwner {...}
function addBot(address bot) external onlyOwner {...}
function removeBot(address bot) external onlyOwner {...}
function setPoolStatus(address pool, PoolStatus status) public onlyOwner {...}
function setBridgeTokens(address[] calldata bridges) external onlyOwner {...}
```

```
function setZeroFeeWallet(address wallet, bool isZeroFee)
external onlyOwner {...}
function setProtocolFeeBps(uint16 newBps) external
onlyOwner {...}
function setBotFeeBps(uint16 newBps) external onlyOwner
{...}
function pause() external onlyGuardian {...}

function unpause() external onlyGuardian {...}
```

```Shell
function setPlanner(address planner_) external onlyOwner
{...}
function setValuation(address valuation_) external
onlyOwner {...}
function setGuardian(address g) external onlyOwner {...}
function setMinimumOpenUSDC(uint256 min_) external
onlyOwner {...}
function setMaxBatchKeys(uint256 maxKeys_) external
onlyOwner {...}
function setSecondPassParams(uint256 thresholdBps_, uint256
minUsdc_) external onlyOwner {...}
function pause() external onlyGuardian {...}

function unpause() external onlyGuardian {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

This risk is accepted for operational flexibility and mitigated in multiple ways: ownership of all contracts will be held by a Timelock contract controlled by a Gnosis Safe multisig, and all administrative actions (aside from pause/unpause) are executed through a minimum 2- day timelock and are fully observable on-chain.

CPI - Cross-Tick Price Impact

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L1047 AerodromeAdapter.sol#L1020
Status	Acknowledged

Description

The `_minOutFromPool` function estimates the expected output of a swap using the time-weighted average price (TWAP), assuming a constant price throughout the transaction:

```shell
uint256 amountInPostFee = FullMath.mulDiv(amountIn,
(1_000_000 - fee), 1_000_000);

uint160 sqrtPriceTwapX96 =
TickMath.getSqrtRatioAtTick(meanTick);
uint256 priceX96 = FullMath.mulDiv(
    uint256(sqrtPriceTwapX96),
    uint256(sqrtPriceTwapX96),
    FixedPoint96.Q96
);

uint256 expectedOut = zeroForOne
    ? FullMath.mulDiv(amountInPostFee, priceX96,
FixedPoint96.Q96)

    : FullMath.mulDiv(amountInPostFee, FixedPoint96.Q96,

priceX96);
```

However, in Uniswap V3's concentrated liquidity model, swaps often traverse multiple price ticks. Each tick crossing shifts the price unfavorably for the swapper, resulting in less output than a simple TWAP-based calculation would predict.

As a result, the `minOut` value may be set too high, causing valid swaps to revert.

Recommendation

Consider using the Quoter contract to simulate the actual swap output, as it accounts for the effects of crossing multiple price ticks.

Team Update

The team has acknowledged that this is not a security issue and states:

We avoid on-chain quoter calls to stay gas-efficient and resist spot manipulation, so tight slippage in volatile markets can cause conservative reverts. Users can widen slippage if they prefer execution over protection.

IDLS - Ineffective Decrease Liquidity Slippage

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L512,631 AerodromeAdapter.sol#L496,615
Status	Acknowledged

Description

Calls to `NPM.decreaseLiquidity` set `amount0Min` and `amount1Min` to zero, effectively disabling the position manager's built-in slippage protection:

```shell
INonfungiblePositionManager.DecreaseLiquidityParams({
    tokenId: tokenId,
    liquidity: lRemove,
    amount0Min: 0,
    amount1Min: 0,
    deadline: block.timestamp

});
```

This pattern is present in both `removeLiquidityBpsUSDC` and `unwindToTokens` functions. While `removeLiquidityBpsUSDC` applies slippage protection during downstream swaps using the `remainingLossUSDC` budget when converting non-USDC tokens, this safeguard does not extend to the liquidity removal step itself. When either token in the pair is USDC, the USDC portion is transferred directly to the receiver without any slippage check.

Furthermore, in the `unwindToTokens` function, no slippage protection is applied at any stage. Tokens are collected and transferred directly to the receiver after liquidity removal.

Recommendation

The team is advised to compute `amount0Min` and `amount1Min` based on the current Time-Weighted Average Price (TWAP) and the amount of liquidity being removed, while applying a reasonable slippage tolerance. This approach ensures that the liquidity removal process is safeguarded against price manipulation, independently of any protections applied to subsequent swaps.

Team Update

The team has acknowledged that this is not a security issue and states:

decreaseLiquidity mins are currently unset to preserve reliable exits and avoid revert-prone burn checks during normal tick movement. User slippage protection remains enforced on all swap/conversion legs via TWAP-based loss-budget checks, and risk is further reduced by strict allowlisting of deep-liquidity pools that are materially harder to manipulate.

PFO - Pool Fee Omission

Criticality	Minor / Informative
Location	Valuation.sol#L649
Status	Acknowledged

Description

The `_quoteThroughPool` function calculates token output amounts using the time-weighted average price (TWAP) without accounting for Uniswap pool fees, which typically range from 0.05% to 1%. As a result, the returned quote reflects a pre-fee value that is higher than what would be received in an actual swap. This leads to a consistent overestimation of token values, and the discrepancy becomes more significant in multi-hop swaps, where fees are applied at each step.

```shell
function _quoteThroughPool(address pool, address tokenIn,
address tokenOut, uint256 amountIn)
    internal
    view
    returns (uint256)
{
    ...

}
```

Recommendation

Consider retrieving the pool's fee tier and subtracting it from the quoted amount. This adjustment ensures that the valuation reflects actual transaction costs, resulting in more accurate and realistic pricing.

Team Update

The team has acknowledged that this is not a security issue and states:

Valuation returns a mid-price estimate and intentionally avoids on-chain quoter or swap simulation to remain cheap and deterministic, so it does not model swap fees, price impact, or tick-crossing. It is used for informational and accounting purposes and for coarse sizing or gating decisions (for example, computing a fee basis). Valuation is not treated as an executable quote.

PTWD - Possible TWAP Window Desync

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L34 UniswapAdapter.sol#L161 AerodromeAdapter.sol#L32 Valuation.sol#L39,81
Status	Acknowledged

Description

Both the adapter and the Valuation contract define their own configurable TWAP (Time-Weighted Average Price) window through separate TWAP_SECONDS variables and independent setter functions:

```shell
// UniswapAdapter.sol
uint32 public TWAP_SECONDS = 60;

function setTwapSeconds(uint32 s) external onlyOwner
{
    if (s < 30 || s > 3600) revert InvalidParam();
    TWAP_SECONDS = s;
}

// Valuation.sol
uint32 public TWAP_SECONDS = 60;

function setTWAPSeconds(uint32 seconds_) external
onlyOwner {
    if (seconds_ < 30 || seconds_ > 3600) revert
InvalidTWAPSeconds();
    TWAP_SECONDS = seconds_;}
```

Because these values are managed independently, updating the TWAP window in one contract without synchronizing it with the other can lead to discrepancies. This results in the adapter and Valuation contract computing prices over different time intervals for the same pool, which may cause inconsistent slippage calculations and inaccurate token valuations.

Recommendation

The team must ensure that the `TWAP_SECONDS` parameter is updated consistently across all relevant contracts. This synchronization is essential to maintain consistent time-weighted average price calculations across the system.

Team Update

The team has acknowledged that this is not a security issue and states:

This is intentional, as it allows fine-tuning of different TWAPs for execution and accounting.

PSM - Price Source Mismatch

Criticality	Minor / Informative
Location	RebalancePlanner.sol#L478 UniswapAdapter.sol#L1047 AerodromeAdapter.sol#L1020
Status	Acknowledged

Description

The `RebalancePlanner` contract calculates swap amounts using the instantaneous spot price retrieved from the `slot0()` function:

```shell
function _safeGetSqrtPrice(address pool) internal view
returns (uint160 sqrtPriceX96) {
    (bool success, bytes memory data) =
pool.staticcall(abi.encodeWithSignature("slot0()"));
    if (!success || data.length < 32) revert Slot0Failed();

    sqrtPriceX96 = abi.decode(data, (uint160));

}
```

In contrast, the `UniswapAdapter` and `AerodromeAdapter` contracts use a time-weighted average price (TWAP) for slippage calculations:

```shell
uint32[] memory secondsAgos = new uint32[](2);
secondsAgos[0] = TWAP_SECONDS;
secondsAgos[1] = 0;

(int56[] memory tickCumulatives, ) =
p.observe(secondsAgos);
```

This inconsistency between using the spot price for planning swaps and the TWAP for slippage protection can lead to issues during periods of high market volatility. When the spot price significantly deviates from the TWAP, the calculated swap amounts may not achieve the intended ratio. As a result, slippage protections may either fail to trigger or permit more slippage than expected.

Recommendation

The team is advised to use a consistent price source across both the `RebalancePlanner` and `UniswapAdapter` components. This ensures accurate swap planning and reliable slippage protection, reducing the risk of discrepancies that could lead to suboptimal trades or unintended losses.

Team Update

The team has acknowledged that this is not a security issue and states:

RebalancePlanner uses the slot0 price because it is meant to be mathematically exact in its calculations so as to avoid leaving dust. Adapters use TWAP to be robust against manipulation when determining slippage limits. These choices are intentionally different, as they serve two distinct purposes.

STW - Short TWAP Window

Criticality	Minor / Informative
Location	Valuation.sol#L39,81
Status	Acknowledged

Description

The contract allows the owner to configure the time-weighted average price (TWAP) interval via the `TWAP_SECONDS` variable, which defaults to 60 seconds and can be set to any value between 30 and 3600 seconds. On Ethereum mainnet, where blocks are produced approximately every 12 seconds, a 60-second TWAP relies on only about five oracle observations. This limited data set makes the price feed vulnerable to manipulation, as a well-capitalized attacker could influence prices over such a short window at a relatively low cost.

In decentralized finance (DeFi), it is common practice to use TWAP intervals of 30 minutes or longer to ensure sufficient resistance against price manipulation.

```shell
uint32 public TWAP_SECONDS = 60;

function setTWAPSeconds(uint32 seconds_) external
onlyOwner {
    if (seconds_ < 30 || seconds_ > 3600) revert
InvalidTWAPSeconds();
    TWAP_SECONDS = seconds_;
    emit TWAPSecondsUpdated(seconds_);

}
```

Recommendation

Consider increasing the minimum Time-Weighted Average Price (TWAP) window. A longer TWAP window reduces the impact of short-term price fluctuations, making it more difficult for attackers to manipulate the average price. This enhances the system's resistance to price manipulation and improves overall price reliability.

Team Update

The team has acknowledged that this is not a security issue and states:

Our TWAP windows are meant to protect against manipulation while still being responsive to volatility. A 30-minute TWAP would misrepresent the current on-chain state for accounting and execution in volatile times. Our 60-second default makes sustained price manipulation extremely difficult on the established high-liquidity pools we support in our allowedPools list.

SCRS - Stale Cached Route Scores

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L278 AerodromeAdapter.sol#L264 Valuation.sol#L267
Status	Acknowledged

Description

When both position tokens are bridge tokens, seedPairFromUSDC selects which bridge to use as the seed token by comparing cached liquidity scores:

```shell
else if (token0IsBridge && token1IsBridge) {
    uint256 score0 =
_bestAllowlistedLiqScore(usdc, token0);
    uint256 score1 =
_bestAllowlistedLiqScore(usdc, token1);
    if (score0 == 0 && score1 == 0) revert
UnsupportedPair();
    seedToken = score0 >= score1 ? token0 :
token1;

}
```

`_bestAllowlistedLiqScore` delegates to Valuation.getBestRoute, which reads from `edgeToConnector`, a cache populated by `refreshAll()` at discrete intervals. If liquidity conditions changed since the last refresh, the comparison may select a bridge with currently inferior liquidity, resulting in worse swap execution.

Recommendation

The team is advised to query current on-chain state instead of relying on cached scores for seed token selection. This will ensure consistency of operations.

Team Update

The team has acknowledged that this is not a security issue and states:

Using cached best liquidity routes is an intentional tradeoff. To derive everything on chain at the time of each getBestRoute call would be prohibitively expensive in gas. refreshAll will be run regularly to ensure up to date best liquidity paths for each route.

UQA - Unconstrained Quoted Amount

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L893,971 AerodromeAdapter.sol#L881,944
Status	Acknowledged

Description

The swap execution functions set the `sqrtPriceLimitX96` parameter to zero when interacting with Uniswap V3 and Aerodrome routers. This parameter defines a price boundary that restricts how far the pool price can move during a swap. By setting it to zero, this safeguard is effectively disabled, allowing swaps to proceed without any constraint on price movement.

```shell
ISwapRouterV3.ExactInputSingleParams memory p =
ISwapRouterV3.ExactInputSingleParams({
    tokenIn: tokenIn,
    tokenOut: tokenOut,
    fee: fee,
    recipient: recipient,
    amountIn: amountIn,
    amountOutMinimum: minOut,
    sqrtPriceLimitX96: 0

});
```

Although the contract enforces slippage protection through the `amountOutMinimum` parameter, the lack of a defined price boundary exposes the swap to potentially significant price impact, as there is no limit on how far the pool price can shift during execution.

Recommendation

Consider whether implementing a price boundary based on the expected Time-Weighted Average Price (TWAP) could offer meaningful additional protection for large swaps, especially in pools with lower liquidity. This approach may help mitigate the impact of price manipulation or sudden volatility.

Team Update

The team has acknowledged that this is not a security issue and states:

User-set minOut already defines the effective price boundary for a swap, so a nonzero sqrtPriceLimitX96 would be redundant.

ZVE - Zero Value Estimation

Criticality	Minor / Informative
Location	Valuation.sol#L219,235,498,649
Status	Acknowledged

Description

The valuation functions are designed to return zero in edge cases, such as when the input amount is extremely small or when quote calculations yield no output. This behavior avoids reverts but introduces a silent failure mode, where a non-zero valuation might be expected but is instead suppressed without warning.

```shell
function usdcValue(address dex, address token, uint256
amount) external view returns (uint256) {
    if (amount == 0) return 0;
    ...

    if (outScaled == 0) return 0;
    ...

}
```

Similarly, internal quote functions also return zero in such scenarios:

```shell
function _quoteThroughPool(...) internal view
returns (uint256) {
    if (amountIn == 0) return 0;
    ...

}
```

This approach can obscure the distinction between an actual zero valuation and a valuation process that failed silently. As a result, downstream logic that depends on these valuations may misinterpret the outcome, potentially leading to incorrect accounting or flawed interpretations.

Recommendation

The team is advised to consider whether critical valuation paths should revert when encountering unexpected zero outputs, rather than returning zero silently. Reverting in such cases allows callers to handle failures explicitly, reducing the risk of misinterpreting a zero return value as a valid result.

Team Update

The team has acknowledged that this is not a security issue and states:

Zero outputs only occur for zero input or sub-dust rounding; all route/TWAP errors revert, so 0 does not mask a failure.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	Valuation.sol#L33,34,39,40,488 UniswapAdapter.sol#L22,23,24,25,26,34 RebalancePlanner.sol#L79,83,86,92,95,97 CLManager.sol#L44,47,50,55,59,60,61,62 CLCore.sol#L46,47 AerodromeAdapter.sol#L22,23,24,25,26,32
Status	Acknowledged

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```shell
address public USDC
address public CORE
uint32 public TWAP_SECONDS = 60
int24 public DEPTH_TICKS = 50


...
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.
Find more information on the Solidity documentation https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
Valuation	Implementation	Ownable, ReentrancyGuard		
		Public	✓	Ownable
	setCore	External	✓	onlyOwner
	setTWAPSeconds	External	✓	onlyOwner
	setDepthTicks	External	✓	onlyOwner
	refreshAll	External	✓	onlyOwner nonReentrant
	_uniqueFactoriesFromPools	Internal		
	usdcValue	External		-
	getBestRoute	External		-
	_bestRouteCached	Internal		
	_connectors	Internal		
	_isConnector	Internal		
	_refreshTokenToConnectors	Internal	✓	
	_bestEdgePoolAndScore	Internal	✓	
	_edgePoolScoreUSDC	Internal		
	_connectorUsdcPool	Internal		
	_edgePoolScoreUSDCExternal	External		-
	_quoteRoute	Internal		

	_poolCandidates	Internal		
	_uniswapPool	Internal		
	_aerodromePool	Internal		
	_poolAllowed	Internal		
	_poolTokens	Internal		
	_ctx	Internal		
	_aerodromeSpacings	Internal		
	_bridgeTokens	Internal		
	_quoteThroughPool	Internal		
	_twapSqrtPriceX96	Internal		
UniswapAdapter	Implementation	Ownable, ReentrancyGuard, Pausable, ICLDexAdapter		
		Public	✓	Ownable
	pause	External	✓	onlyGuardian
	unpause	External	✓	onlyGuardian
	setManager	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	setCore	External	✓	onlyOwner
	setTwapSeconds	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	getQuoter	External		-
	getNPM	External		-
	getFactory	External		-

	isAerodrome	External		-
	bestAllowlistedLiqScore	External		-
	validateAndGetPoolParams	External		-
	seedPairFromUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	mintPosition	External	✓	nonReentrant whenNotPaused onlyManager
	addLiquidity	External	✓	nonReentrant whenNotPaused onlyManager
	removeLiquidityBpsUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToTokens	External	✓	nonReentrant whenNotPaused onlyManager
	unwindToTokens	External	✓	nonReentrant whenNotPaused onlyManager
	swapExactUSDCForToken	External	✓	nonReentrant whenNotPaused onlyManager
	swapExactInToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	quoteToToken	External	✓	-
	getExpectedOutUSDC	External		-
	_getBestPath	Internal		
	_swapUSDCToToken	Private	✓	
	_swapTokenToUSDC	Private	✓	
	_swapExactInSingleHop	Private	✓	
	_executeRebalance	Private	✓	

	_quoteExactInSingleHop	Private	✓	
	_bestFee	Internal		
	_poolFor	Internal		
	_poolFee	Internal		
	_getTokensFromPosition	Internal		
	_poolTokens	Internal		
	_counterpartToken	Internal		
	_minOutFromPool	Internal		
	_minOutFromExpected	Internal		
	_expectedOutFromPool	Internal		
	_computeSlippageMins	Internal		
	_bestAllowlistedLiqScore	Internal		
	_requireBalance	Internal		
	_poolAllowed	Internal		
	_isBridgeToken	Internal		
	_usdcValue	Internal		
	_expectedOutUSDC	Internal		
	_lossBpsFromBudget	Internal		
	_minOutFromLossBudget	Internal		
	_usedLossUSDC	Internal		
	_clampSlippage	Internal		
	_approveExact	Internal	✓	
	_convertDustToUSDC	Internal	✓	
	_returnDust	Internal	✓	
	_collectFeesToTokensInternal	Internal	✓	

RebalancePlanner	Implementation	Ownable		
		Public	✓	Ownable
	setCore	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	setSolverIterations	External	✓	onlyOwner
	setEarlyStop	External	✓	onlyOwner
	planFromTokenBundle	External		-
	_solveIterative	Internal		
	_loadPoolContext	Internal		
	_safeGetSqrtPrice	Internal		
	_poolAllowed	Internal		
	_isBridgeToken	Internal		
ProtocolReserve	Implementation	Ownable, Pausable, ReentrancyGuard		
		Public	✓	Ownable
	pause	External	✓	onlyOwner
	unpause	External	✓	onlyOwner
	usdcBalance	External		-
	getShares	External		-
	getShareholders	External		-
	setShares	External	✓	onlyOwner whenNotPaused

	sweepReserves	External	✓	onlyOwner whenNotPaused nonReentrant
ICLDexAdapter	Interface			
	validateAndGetPoolParams	External		-
	seedPairFromUSDC	External	✓	-
	mintPosition	External	✓	-
	addLiquidity	External	✓	-
	removeLiquidityBpsUSDC	External	✓	-
	getExpectedOutUSDC	External		-
	unwindToTokens	External	✓	-
	swapExactUSDCForToken	External	✓	-
	swapExactInToUSDC	External	✓	-
	collectFeesToUSDC	External	✓	-
	collectFeesToTokens	External	✓	-
	quoteToToken	External	✓	-
	bestAllowlistedLiqScore	External		-
	getNPM	External		-
	getQuoter	External		-
	getFactory	External		-
	isAerodrome	External		-
ICLCore	Interface			
	isPoolAllowed	External		-
	isBridgeToken	External		-

	bridgeTokens	External		-
	listAllowedPools	External		-
	allowedDexes	External		-
	listAllowedDexes	External		-
	positionValueUSDC	External		-
	positionValueUSDCSingle	External		-
	pendingFees	External		-
	getPositionDetails	External		-
	getPosition	External		-
	USDC	External		-
	VALUATION	External		-
	protocolFeeBps	External		-
	botFeeBps	External		-
	protocolReserve	External		-
	positions	External		-
	allowedBots	External		-
	zeroFeeWallets	External		-
	isPoolDeprecated	External		-
	registerPosition	External	✓	-
	deregisterPosition	External	✓	-
	updateTokenMetadata	External	✓	-
	adjustTotalDeposited	External	✓	-
	setBotAllowedForPosition	External	✓	-
	addDustToPosition	External	✓	-
	withdrawDustForPosition	External	✓	-

	returnPosition	External	✓	-
	spotAmounts	External		-
ICLManager	Interface			
	protocolFeeBps	External		-
	botFeeBps	External		-
IValuation	Interface			
	usdcValue	External		-
	getBestRoute	External		-
IRebalancePlanner	Interface			
	planFromTokenBundle	External		-
INonfungiblePositionManager	Interface			
	positions	External		-
	mint	External	✓	-
	increaseLiquidity	External	✓	-
	decreaseLiquidity	External	✓	-
	collect	External	✓	-
	burn	External	✓	-
	ownerOf	External		-
	safeTransferFrom	External	✓	-
	transferFrom	External	✓	-

	setApprovalForAll	External	✓	-
ISlipstreamNonfungiblePositionManager	Interface			
	positions	External		-
	mint	External	✓	-
	increaseLiquidity	External	✓	-
	decreaseLiquidity	External	✓	-
	collect	External	✓	-
	burn	External	✓	-
	ownerOf	External		-
	safeTransferFrom	External	✓	-
	setApprovalForAll	External	✓	-
IUniswapV3Factory	Interface			
	getPool	External		-
IAerodromeFactory	Interface			
	getPool	External		-
	tickSpacings	External		-
	tickSpacingToFee	External		-
IUniswapV3Pool	Interface			
	liquidity	External		-

	token0	External		-
	token1	External		-
	fee	External		-
	tickSpacing	External		-
	factory	External		-
	slot0	External		-
	observe	External		-
	ticks	External		-
	feeGrowthGlobal0X128	External		-
	feeGrowthGlobal1X128	External		-
ICLPool	Interface	IUniswapV3 Pool		
ISlipstreamPool	Interface			
	slot0	External		-
ISwapRouterV3	Interface			
	exactInputSingle	External	Payable	-
ISlipstreamSwapRouter	Interface			
	exactInputSingle	External	Payable	-
IUniswapQuoterV2Like	Interface			
	quoteExactInputSingle	External	✓	-

ISlipstreamQuoterV2Like	Interface			
	quoteExactInputSingle	External	✓	-
ISlipstreamPoolState	Interface			
	feeGrowthGlobal0X128	External		-
	feeGrowthGlobal1X128	External		-
	ticks	External		-
	liquidity	External		-
CLManager	Implementation	ReentrancyGuard, Pausable, Ownable		
		Public	✓	Ownable
	setPlanner	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	setMinimumOpenUSDC	External	✓	onlyOwner
	setMaxBatchKeys	External	✓	onlyOwner
	setSecondPassParams	External	✓	onlyOwner
	pause	External	✓	onlyGuardian
	unpause	External	✓	onlyGuardian
	_requireOwnerOrBot	Internal		
	openPosition	External	✓	nonReentrant whenNotPaused

	exitPosition	External	✓	nonReentrant whenNotPaused
	returnNft	External	✓	nonReentrant
	collectFeesToUSDC	External	✓	nonReentrant whenNotPaused
	compoundFees	External	✓	nonReentrant whenNotPaused
	_compoundOne	Internal	✓	
	addCollateral	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	removeCollateral	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	changeRange	External	✓	nonReentrant onlyKeyOwner OrBot whenNotPaused
	allowBotForPosition	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	withdrawDust	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	_addDust	Internal	✓	
	_quotedPositionValue	Internal		
	_corePositionValue	Internal		
	_tokenValueUSDC	Internal		
	_isZeroFee	Internal		
	_protocolFeeConfig	Internal		
	_protocolFeeAmount	Internal		

	_lossBudgetUSDC	Internal		
	_maybeSecondPass	Internal	✓	
	_handleBotFeeOnCollected	Internal	✓	
CLCore	Implementation	Ownable, IERC721Receiver, Pausable, ReentrancyGuard, ICLCore		
		Public	✓	Ownable
	setValuation	External	✓	onlyOwner
	setProtocolReserve	External	✓	onlyOwner
	setManager	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	addAllowedDex	External	✓	onlyOwner
	removeAllowedDex	External	✓	onlyOwner
	listAllowedDexes	External		-
	addBot	External	✓	onlyOwner
	removeBot	External	✓	onlyOwner
	setPoolStatus	Public	✓	onlyOwner
	setBridgeTokens	External	✓	onlyOwner
	setZeroFeeWallet	External	✓	onlyOwner
	setProtocolFeeBps	External	✓	onlyOwner
	setBotFeeBps	External	✓	onlyOwner
	pause	External	✓	onlyGuardian
	unpause	External	✓	onlyGuardian

	registerPosition	External	✓	nonReentrant onlyManager whenNotPaused
	deregisterPosition	External	✓	nonReentrant onlyManager whenNotPaused
	addDustToPosition	External	✓	nonReentrant onlyManager whenNotPaused
	withdrawDustForPosition	External	✓	nonReentrant onlyManager whenNotPaused
	returnPosition	External	✓	nonReentrant onlyManager
	updateTokenMetadata	External	✓	nonReentrant onlyManager whenNotPaused
	adjustTotalDeposited	External	✓	nonReentrant onlyManager whenNotPaused
	setBotAllowedForPosition	External	✓	nonReentrant onlyManager whenNotPaused
	isPoolAllowed	External		-
	listAllowedPools	External		-
	bridgeTokens	External		-
	isBridgeToken	Public		-
	listUserPositionKeys	External		-
	_nextKey	Internal	✓	
	getPosition	External		-
	getPositionDetails	Public		-
	listUserPositionDetails	External		-

	positionValueUSDC	External		-
	positionValueUSDCSingle	External		-
	spotAmounts	External		-
	pendingFees	External		-
	_pendingFeesForPosition	Internal		
	_positionValueUSDCSingle	Internal		
	_subUint256	Internal		
	_pendingFeesCalc	Internal		
	_registerPosition	Internal	✓	
	_deregisterPosition	Internal	✓	
	_positionPool	Internal		
	_spotAmountsForPosition	Internal		
	_liquidityOf	Internal		
	_pool	Internal		
	isPoolDeprecated	External		-
	_isPoolAllowed	Internal		
	_readSlot0	Internal		
	_valueTokenToUSDC	Internal		
	onERC721Received	External		-
AerodromeAdapter	Implementation	Ownable, ReentrancyGuard, Pausable, ICLDexAdapter		
		Public	✓	Ownable
	pause	External	✓	onlyGuardian

	unpause	External	✓	onlyGuardian
	setManager	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	setCore	External	✓	onlyOwner
	setTwapSeconds	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	getQuoter	External		-
	getNPM	External		-
	getFactory	External		-
	isAerodrome	External		-
	bestAllowlistedLiqScore	External		-
	validateAndGetPoolParams	External		-
	seedPairFromUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	mintPosition	External	✓	nonReentrant whenNotPaused onlyManager
	addLiquidity	External	✓	nonReentrant whenNotPaused onlyManager
	removeLiquidityBpsUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToTokens	External	✓	nonReentrant whenNotPaused onlyManager
	unwindToTokens	External	✓	nonReentrant whenNotPaused onlyManager

	swapExactUSDCForToken	External	✓	nonReentrant whenNotPaused onlyManager
	swapExactInToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	quoteToToken	External	✓	-
	getExpectedOutUSDC	External		-
	_getBestPath	Internal		
	_swapUSDCToToken	Private	✓	
	_swapTokenToUSDC	Private	✓	
	_swapExactInSingleHop	Private	✓	
	_executeRebalance	Private	✓	
	_quoteExactInSingleHop	Private	✓	
	_bestTickSpacing	Internal		
	_poolFor	Internal		
	_poolTickSpacing	Internal		
	_getTokensFromPosition	Internal		
	_poolTokens	Internal		
	_counterpartToken	Internal		
	_minOutFromPool	Internal		
	_minOutFromExpected	Internal		
	_expectedOutFromPool	Internal		
	_computeSlippageMins	Internal		
	_bestAllowlistedLiqScore	Internal		
	_requireBalance	Internal		
	_poolAllowed	Internal		

	_isBridgeToken	Internal		
	_usdcValue	Internal		
	_expectedOutUSDC	Internal		
	_lossBpsFromBudget	Internal		
	_minOutFromLossBudget	Internal		
	_usedLossUSDC	Internal		
	_clampSlippage	Internal		
	_approveExact	Internal	✓	
	_convertDustToUSDC	Internal	✓	
	_returnDust	Internal	✓	
	_collectFeesToTokensInternal	Internal	✓	

Inheritance Graph

The inheritance graph for the EZManager contracts can be accessed through the link.

Flow Graph

The flow graph for the EZManager contracts can be accessed through the link.

Summary

The EZManager protocol proposes a liquidity management system, providing modular tracking, accounting, and valuation of LP positions across Uniswap V3 and Aerodrome Slipstream. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io